MUSTAFACODE

Lecture 10

this Pointer, const Member Functions, Complex Number & Interface Separation (C++)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

CS304 OBJECT-ORIENTED PROGRAMMING

Lecture 10: this Pointer, const Member Functions, Complex Number & Interface Separation (C++)

📘 Chapter 10 — this Pointer, const Member Functions, Complex Number & Interface Separation (C++)

🔹 10.1 Uses of `this` Pointer

📌 What is `this` pointer?

`this` pointer ek hidden pointer hota hai jo **current object ka address hold karta hai**.

👉 Jab bhi object ka function call hota hai, compiler automatically `this` pointer pass karta hai.

🔹 Important Use: Return Current Object

Kabhi kabhi hum chahte hain ke function **current object return kare**, taa ke chaining ho sake.

👉 Is case mein hum use karte hain:

```
return *this;
```

📌 Example

```
class Student {
    int rollNo;
    string name;

public:
    Student setName(string n) {
        name = n;
        return *this;
    }

    Student setRollNo(int r) {
        rollNo = r;
        return *this;
    }
};
```

📌 Usage (Method Chaining)

```
int main() {
    Student aStudent, bStudent;

    bStudent = aStudent.setName("Ahmad").setRollNo(10);

    return 0;
}
```

🔹 Real Life Example

👉 Banking app:

```
account.deposit(1000).withdraw(500).checkBalance();
```

📌 Key Idea:

- `this` = current object

- `*this` = current object itself
- Used for **method chaining**

🔹 10.2 Separation of Interface & Implementation

📌 Interface kya hota hai?

Class ke **public functions** jo user ko dikhte hain.

📌 Implementation kya hota hai?

Function ka actual code jo hidden hota hai.

🔹 Why separation is important?

- Code clean hota hai
- Implementation change ho sakti hai without affecting users
- Maintenance easy hota hai

📌 Real Life Example

👉 Mobile phone:

- Interface = buttons / screen
- Implementation = internal circuits

User ko sirf buttons nazar aate hain.

📌 Complex Number Example

🔹 Old Implementation

```
class Complex {
    float x;
    float y;

public:
    void setNumber(float i, float j) {
        x = i;
        y = j;
    }

    float getX() { return x; }
    float getY() { return y; }
};
```

🔹 New Implementation

```
class Complex {
    float z;
    float theta;

public:
    void setNumber(float i, float j) {
        z = sqrt(i*i + j*j);
        theta = atan(j/i);
    }
};
```

📌 Key Difference

Old	New
x, y store	z, theta store
direct form	polar form
less flexible	more flexible

📌 Advantages

- Internal structure hidden
- Implementation change possible
- User code unaffected

🔷 Interface vs Implementation in C++

📌 Header File (.h) → Interface

```
class Student {
    int rollNo;

public:
    void setRollNo(int);
    int getRollNo();
};
```

📌 Source File (.cpp) → Implementation

```cpp
#include "Student.h"

void Student::setRollNo(int r) {
    rollNo = r;
}

int Student::getRollNo() {
    return rollNo;
}
```

📌 Main File

```cpp
#include "Student.h"

int main() {
    Student s;
    s.setRollNo(10);
}
```

🔹 Simple Concept

- `.h` = Design (what to do)
- `.cpp` = Working (how to do)
- `main.cpp` = Use

🔹 10.4 const Member Functions

📌 What is const function?

Aisa function jo **object ka data change nahi kar sakta**

🔹 Syntax

```
class Student {
public:
    int getRollNo() const;
};
```

🔹 Definition

```
int Student::getRollNo() const {
    return rollNo;
}
```

📌 Real Life Example

👉 Bank balance check function:

- Sirf balance show kare
- Change na kare

🔹 Why const is important?

- Bugs prevent hotay hain
- Data safe rehta hai
- Accidental modification avoid hoti hai

❌ Bug Example

```
bool Student::isRollNo(int aNo) {
    if (rollNo = aNo) { // mistake
        return true;
    }
    return false;
}
```

🔥 Correct Version

```
bool Student::isRollNo(int aNo) const {
    if (rollNo == aNo) {
        return true;
    }
    return false;
}
```

📌 Important Rules

- const function data change nahi karta
- non-const functions call nahi kar sakta

❌ Constructor/Destructor const nahi ho sakte

```
class Time {
public:
    Time() const {}   // ❌
    ~Time() const {}  // ❌
};
```

🔷 10.5 this Pointer in const Functions

📌 Normal function

```
Student * const this;
```

📌 const function

```
const Student * const this;
```

🔹 Simple Table

Type	this pointer behavior
Normal	editable object
const	read-only object

📘 FINAL SUMMARY

🔹 this pointer

- current object refer karta hai
- chaining possible karta hai

🔹 const function

- object protect karta hai
- read-only access deta hai

🔹 Interface separation

- .h = interface
- .cpp = implementation
- flexibility increase karta hai

🔹 Complex number concept

- old: x, y
- new: z, theta
- internal change safe hota hai

🎯 EXAM TIP

Agar question aaye:

👉 this pointer + const function + interface separation

toh likho:

- definition
- example
- advantage
- real life example